SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: **Total Marks:100**

Annexure A

Debugging

Code of Conduct:

I ____________________ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q. No	Understanding of Debugging Concepts (25)	Error Identification(25)	Root Causes Analysis (20)	Error Corrections and Solution Design(20)	Testing and Validation(10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/ 25**	**/ 20**	**/ 20**	**/ 10**	**/ 100**

Annexure A

Debugging

Q1 Consider the following syntax-directed definition:

```
E → E1 + T  ( E.val = T.val }

E → T       { E.val = T.val }

T → num     { T.val = num.lexval }
```

For the input:

```
3 + 5
```

the compiler produces:

```
E.val = 5
```

Debug the given syntax-directed definition. Identify the incorrect semantic rule, explain its effect on attribute evaluation, provide the corrected rule, and determine the correct value of the expression.

Q2 . For the expression:

```
a + b * c
```

the compiler constructs the following syntax tree:

```
        *
       / \
      +   c
     / \
    a   b
```

The compiler subsequently generates:

```
t1 = a + b
t2 = t1 * c
```

Debug the syntax-tree construction. Analyze the relationship between operator precedence and the generated tree, construct the corrected syntax tree, and generate the correct intermediate representation.

Q3. For the expression:

```
3 + 4 * 5
```

using:

```
E → E + T | T
T → T * F | F
F → num
```

the compiler annotates the root with:

```
E.val = 35
```

Analyze the annotated parse tree. Trace the synthesized attributes from the leaves to the root, identify the incorrect computation, and determine the correct value of the expression.

Q4 . The symbol table contains:

Identifier	Type
a	int
b	float

Identifier	Type
c	int
x	float

For the statement:

```
x = a + b * c;
```

the compiler reports:

```
Type Error: incompatible operands
```

Debug the type-checking process. Analyze each subexpression, determine the type of `b * c` and `a + (b * c)`, identify where type conversion is required, and determine whether the assignment to `x` is valid.

Q5. Consider:

```
int a;
float b;
int x;
```

and the statements:

```
x = a + b;
b = a + x;
```

The type checker reports both statements as:

```
Valid
```

and does not generate any conversion.

Analyze the type-checking results. Determine the type of each expression, identify the required conversions, and explain how the type checker should process each assignment.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Debugging Concepts	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Error Identification	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Root Causes Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Error Correction & Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Testing and Validation	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand